

MÓDULO 9

Testing y Debugging Asistidos por IA

Unit · Integración · Smoke · Seguridad · E2E · Debugging · CI Auto-fix

Plan de Formación — OpenAI Codex para Desarrolladores

Nivel: Intermedio-Avanzado · Duración estimada: 3–4 horas

Versión 1.0 — Febrero 2026

Índice

Índice.....	2
1. Introducción.....	4
2. Objetivos de aprendizaje	4
3. Contenidos teóricos	5
3.1 Pirámide de pruebas con Codex.....	5
3.2 Workflow oficial de testing con Codex	5
3.3 Workflow oficial de debugging con Codex.....	5
3.4 Auto-fix de tests fallidos en CI.....	6
3.5 Checks de seguridad SAST con Codex.....	6
3.6 Tests de caracterización (Snapshot Testing)	7
4. Buenas prácticas	8
4.1 Scope explícito	8
4.2 Pedir edge cases explícitos	8
4.3 Exigir ejecución	8
4.4 Tests antes del fix	8
4.5 Seguir convenciones	8
4.6 Minimizar flakiness (E2E).....	8
5. Errores comunes.....	9
5.1 Tests placebo	9
5.2 E2E excesivos	9
5.3 Debugging sin repro steps	9
5.4 Tests sin teardown	9
5.5 Aceptar tests sin ejecutarlos	9
6. Casos de uso reales	10
6.1 Bug intermitente en producción	10
6.2 Endurecer validación de inputs	10
6.3 Auto-fix en CI.....	10
6.4 SAST + Codex en GitLab.....	10
7. Laboratorio práctico (L9) — Batería de Pruebas Completa	11
7.1 Paso 1 — Crear proyecto base.....	11
7.2 Paso 2 — Crear módulo auth con bugs.....	11
7.3 Paso 3 — AGENTS.md con reglas de testing	12
7.4 Paso 4 — Generar tests UNITARIOS.....	12
7.5 Paso 5 — Generar tests de INTEGRACIÓN	13
7.6 Paso 6 — Generar tests de SEGURIDAD	13
7.7 Paso 7 — Generar smoke tests	14

7.8 Paso 8 — Debugging: corregir bugs.....	14
7.9 Paso 9 — Coverage y verificación final	14
7.10 Limpieza (Protocolo Obligatorio).....	15
8. Resumen y siguientes pasos.....	16
8.2 Preparación para el Módulo 10	16
9. Referencias	16

1. Introducción

Codex puede generar tests en todos los niveles de la pirámide de pruebas, pero la calidad del test depende directamente de la calidad del prompt. Un prompt genérico como "genera tests" produce "tests placebo" que pasan siempre pero no detectan nada. En este módulo aprenderemos a escribir prompts específicos que generan tests útiles, a usar el patrón oficial de debugging de Codex, y a automatizar testing en CI/CD.

2. Objetivos de aprendizaje

#	Objetivo	Evidencia de logro
O1	Generar suites de tests útiles con edge cases en todos los niveles de la pirámide.	Tests unitarios, integración, smoke y seguridad generados con coverage >80%.
O2	Debugging basado en reproducción con el patrón oficial de Codex.	Bug reproducido con test que falla → fix → test pasa.
O3	Automatizar tests en CI con codex exec y GitHub Action.	Conocer el workflow auto-fix de tests fallidos.

3. Contenidos teóricos

3.1 Pirámide de pruebas con Codex

Codex puede generar tests en todos los niveles. La clave es el prompt: cuanto más específico el scope y los edge cases, mejor será el test generado.

Nivel	Qué testea	Prompt pattern
Unit	Lógica pura, contratos	"Write a unit test for this function. Cover happy path plus edge cases."
Integración	DB, colas, IO, APIs	"Generate integration tests using mocks/fixtures for the DB layer. Include setup/teardown."
Smoke	Rutas críticas post-deploy	"Create a smoke test that verifies the health endpoint and auth flow."
Seguridad	SAST, inputs, authz	"Analyze for SQL injection, XSS, path traversal, hardcoded secrets."
E2E/GUI	Flujos clave	"Generate E2E tests with Playwright. Use data-testid selectors."

3.2 Workflow oficial de testing con Codex

El workflow documentado por OpenAI para generar tests sigue estos pasos:

- Abrir el fichero con la función a testear (o usar `@path/to/file.py` en CLI).
- Prompt específico con scope explícito: función, edge cases, convenciones.
- Codex genera el test y lo ejecuta (en `--full-auto` o con aprobación).
- Verificar que los tests pasan (green) y cubren lo pedido.

Ejemplo de prompt para testing (del workflow oficial):

```
Write a unit test for the process_payment function
in @src/payments.py.
Cover:
- happy path (valid card, sufficient funds)
- edge cases: expired card, zero amount, negative amount
- error path: network timeout, invalid currency
Follow conventions used in other tests in @tests/.
```

💡 Clave: scope explícito

"Add a test for the invert_list function in @transform.ts. Cover the happy path plus edge cases."
— Este es el patrón oficial de OpenAI para testing con Codex.

3.3 Workflow oficial de debugging con Codex

OpenAI documenta un patrón específico para debugging que incluye cuatro elementos:

- Repro steps: Pasos exactos para reproducir el bug.
- Expected vs Actual: Qué debería pasar vs. qué pasa.

- Constraints: Qué NO tocar (API shape, DB schema, etc.).
- Verification: Pedir que re-ejecute los repro steps después del fix.

Ejemplo del workflow oficial de debugging:

```
Bug: Clicking "Save" on settings sometimes shows
"Saved" but doesn't persist the change.

Repro:
1) Start the app: npm run dev
2) Go to /settings
3) Toggle "Enable alerts"
4) Click Save
5) Refresh: the toggle resets

Constraints:
- Do not change the API shape.
- Keep the fix minimal and add a regression test.

Start by reproducing the bug locally, then propose
a patch and run checks.
```

Codex descubre contexto

Codex ejecuta los repro steps, descubre call sites y stack traces, propone un patch y ejecuta verificaciones. El prompt proporciona la estructura; Codex descubre el contexto.

3.4 Auto-fix de tests fallidos en CI

Codex puede corregir automáticamente tests fallidos en CI/CD usando codex exec en un GitHub Action. El cookbook oficial documenta un workflow que:

- Detecta cuando el CI falla (workflow_run con conclusion == failure).
- Hace checkout del commit que falló.
- Ejecuta codex exec con un prompt de fix mínimo y sandbox workspace-write.
- Re-ejecuta los tests para verificar.
- Abre un PR automático con la corrección.

```
# Prompt del auto-fix oficial:
codex exec --full-auto --sandbox workspace-write \
  "Read the repository, run the test suite,
  identify the minimal change needed to make
  all tests pass, implement only that change.
  Do not refactor unrelated files."
```

3.5 Checks de seguridad SAST con Codex

El cookbook oficial de GitLab muestra cómo Codex complementa herramientas SAST tradicionales:

- Code Quality Reports: Generar JSON CodeClimate para CI.
- Vulnerability Interpretation: Transformar output de scanners en planes priorizados.
- Automated Remediation: Generar patches .patch validados con git apply --check.

⚠ Codex complementa, no reemplaza

Codex no sustituye scanners SAST como Semgrep o Bandit. Añade interpretación, priorización y generación de patches sobre sus resultados.

3.6 Tests de caracterización (Snapshot Testing)

Antes de tocar código legacy sin tests, pedir a Codex que capture el comportamiento actual:

```
Generate a characterization test that captures the
CURRENT behavior of @src/legacy_module.py, even if
the behavior is incorrect.
I want to ensure my refactor doesn't change the output.
Do NOT fix any bugs yet — only capture what exists.
```

Esto crea una "red de seguridad" antes del refactor (ver Módulo 8).

4. Buenas prácticas

4.1 Scope explícito

"Write a unit test for THIS function" es mejor que "write tests for the project". Codex genera mejor con scope acotado.

4.2 Pedir edge cases explícitos

"Cover: empty input, None, negative numbers, max int, unicode, concurrent access." Sin edge cases explícitos, Codex genera solo happy path.

4.3 Exigir ejecución

En --full-auto, Codex ejecuta pytest/npm test automáticamente. Verificar que los tests pasan (green) antes de aceptar.

4.4 Tests antes del fix

Para debugging: primero "Write a test that reproduces this bug (should FAIL)", después "Fix the bug and verify the test passes."

4.5 Seguir convenciones

Incluir @tests/ en el prompt para que Codex siga las convenciones del proyecto. Definir reglas en AGENTS.md.

4.6 Minimizar flakiness (E2E)

Pedir waits explícitos (waitForSelector), datos deterministas (fixtures), selectores estables (data-testid).

5. Errores comunes

5.1 Tests placebo

Problema: El test copia la lógica del código en vez de verificar el contrato.

Solución: "Test the CONTRACT, not the implementation. The test should still pass if the internals change."

5.2 E2E excesivos

Problema: Usar E2E para validar lógica que debería ser unit test.

Solución: Respetar la pirámide: unit > integración > E2E.

5.3 Debugging sin repro steps

Problema: "Fix this bug" sin pasos de reproducción. Codex no puede reproducir lo que no puede ejecutar.

Solución: Incluir repro steps, expected vs actual, constraints (patrón oficial de debugging).

5.4 Tests sin teardown

Problema: Tests de integración que dejan estado (ficheros, DB, procesos).

Solución: Pedir fixtures con setup/teardown explícito.

5.5 Aceptar tests sin ejecutarlos

Problema: Codex genera tests que parecen correctos pero no corren.

Solución: Siempre usar --full-auto o pedir "Run the tests and show the output."

6. Casos de uso reales

6.1 Bug intermitente en producción

Proporcionar logs y trazas a Codex. Pedir que analice, identifique root cause, escriba test que reproduzca el bug determinísticamente, y lo corrija.

6.2 Endurecer validación de inputs

Usar /review (M8) para identificar inputs no validados, luego generar tests de seguridad que intenten SQL injection, XSS, path traversal e integer overflow.

6.3 Auto-fix en CI

GitHub Action que detecta tests fallidos, ejecuta codex exec con prompt de fix mínimo, y abre PR automático. Documentado en el cookbook oficial de OpenAI.

6.4 SAST + Codex en GitLab

Pipeline que ejecuta scanner SAST, pasa el report a Codex para interpretación y generación de patches de remediación. Probado con OWASP Juice Shop.

7. Laboratorio práctico (L9) — Batería de Pruebas Completa

🎯 Objetivo del laboratorio

Crear un módulo de autenticación con 7 bugs deliberados, generar tests en todos los niveles de la pirámide (unit, integración, smoke, seguridad), usar Codex para debugging y verificar que los tests detectan los problemas.

7.1 Paso 1 — Crear proyecto base

Tiempo estimado: 5 minutos

```
mkdir /tmp/codex-lab09 && cd /tmp/codex-lab09
git init

python3 -m venv .venv
source .venv/bin/activate
pip install pytest pytest-cov flake8 httpx

cat > .gitignore << 'EOF'
.venv/
__pycache__/
*.pyc
*.db
.coverage
EOF

mkdir -p src tests
```

7.2 Paso 2 — Crear módulo auth con bugs

Tiempo estimado: 5 minutos

El módulo incluye 7 bugs deliberados: salt fijo, sin validación de username/password, SQL injection en role, sin rate limiting, None handling, reset sin autenticación.

```
cat > src/auth.py << 'PYEOF'
"""Módulo de autenticación — con bugs deliberados."""
import hashlib, sqlite3, os

DB_PATH = "auth.db"

def init_db():
    conn = sqlite3.connect(DB_PATH)
    conn.execute("""CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY, username TEXT UNIQUE NOT NULL,
        password_hash TEXT NOT NULL, role TEXT DEFAULT 'user',
        login_attempts INTEGER DEFAULT 0)""")
    conn.commit(); conn.close()

def hash_password(password: str) -> str:
    # BUG 1: Salt fijo
```

```

    salt = "fixed_salt_123"
    return hashlib.sha256((salt + password).encode()).hexdigest()

def register(username, password, role="user"):
    # BUG 2-3: Sin validación username/password
    # BUG 4: SQL injection en role (f-string)
    conn = sqlite3.connect(DB_PATH)
    hashed = hash_password(password)
    conn.execute(f"INSERT INTO users ... VALUES (?, ?, '{role}']",
        (username, hashed))
    conn.commit(); conn.close()

def login(username, password) -> dict:
    # BUG 5: Sin rate limiting
    ...

def is_admin(username) -> bool:
    # BUG 6: Sin manejo de None
    ...

def reset_password(username, new_password):
    # BUG 7: Sin verificación de identidad
    ...
PYEOF

git add -A && git commit -m "chore: módulo auth con bugs"

```

7.3 Paso 3 — AGENTS.md con reglas de testing

```

cat > AGENTS.md << 'EOF'
# Project Instructions
## Testing Rules
- Always run pytest after generating tests.
- Use fixtures with setup/teardown for DB tests.
- Test the CONTRACT, not the implementation.
- Include edge cases: empty, None, special chars, SQL injection.
- Organize: tests/unit/, tests/integration/, tests/smoke/, tests/security/
- Use conftest.py for shared fixtures.
## Debugging Rules
- FIRST write a test that reproduces the bug (FAIL),
  then fix, then verify (PASS).
EOF

git add AGENTS.md && git commit -m "docs: AGENTS.md"

```

7.4 Paso 4 — Generar tests UNITARIOS

Tiempo estimado: 10 minutos

```
codex --full-auto
```

Prompt en el TUI:

```

Genera tests unitarios para @src/auth.py
en tests/unit/test_auth_unit.py.

Cubre para CADA función:
- happy path
- edge cases: username vacío, password vacía,
  caracteres especiales, unicode
- boundary: password de 1 char, de 1000 chars
- error: usuario duplicado, usuario no existe

Crea conftest.py con fixture que inicialice
y limpie la DB antes de cada test.
Ejecuta pytest tests/unit/ -v.

```

7.5 Paso 5 — Generar tests de INTEGRACIÓN

```

Genera tests de integración en
tests/integration/test_auth_integration.py.

Flujos completos:
1. register → login → is_admin
2. register → login incorrecto → reintentos
3. register → reset_password → login nuevo
4. Dos usuarios, cada uno con su role

Ejecuta pytest tests/integration/ -v.

```

7.6 Paso 6 — Generar tests de SEGURIDAD

```

Genera tests de seguridad en
tests/security/test_auth_security.py.

Intenta explotar CADA vulnerabilidad:
1. SQL injection en parámetro role de register()
2. Brute force: login() 100 veces sin bloqueo
3. Salt fijo: misma password = mismo hash (DEBE FALLAR)
4. reset_password sin auth (DEBE FALLAR)
5. register con username vacío o caracteres de control

Cada test con docstring explicando qué vulnerabilidad
intenta explotar. Ejecuta pytest tests/security/ -v.

```

💡 Tests que DEBEN fallar

Los tests de seguridad están diseñados para fallar, revelando que las vulnerabilidades son reales. Esto es el objetivo: demostrar que los bugs existen antes de corregirlos.

7.7 Paso 7 — Generar smoke tests

```
Genera smoke test en tests/smoke/test_smoke.py:
1. init_db() no lanza excepciones
2. register + login funciona (happy path)
3. is_admin retorna False para usuario normal
```

Este smoke debe PASAR incluso con los bugs.
Ejecuta `pytest tests/smoke/ -v`.

7.8 Paso 8 — Debugging: corregir bugs

Tiempo estimado: 15 minutos

Analiza los tests que fallaron. Para CADA bug:

1. Identifica causa raíz.
2. Verifica que existe test que lo reproduce.
3. Corrige en `@src/auth.py`:
 - BUG 1: `os.urandom(16)` random salt
 - BUG 2-3: validar username/password
 - BUG 4: parámetro (?) en vez de f-string
 - BUG 5: check de `login_attempts`
 - BUG 6: manejar `None` en `is_admin`
 - BUG 7: requerir password actual en `reset`
4. Ejecuta `pytest --tb=short` para verificar.

NO modifiques los tests: son el contrato.

7.9 Paso 9 — Coverage y verificación final

```
PYTHONPATH=. pytest tests/ -v --tb=short \
  --cov=src --cov-report=term-missing
flake8 src/ tests/
```

Verificación	Resultado esperado
Tests unitarios generados	tests/unit/ con edge cases para cada función
Tests integración generados	tests/integration/ con flujos completos
Tests seguridad generados	tests/security/ con exploits reales
Smoke tests generados	tests/smoke/ con rutas críticas
Tests seguridad detectaron bugs	Salt fijo, SQL injection, sin auth...
Bugs corregidos	Todos los tests pasan (green)
Coverage	>80% de <code>src/auth.py</code>
Lint limpio	flake8 sin errores
Patrón test-first	Tests existían antes de los fixes

7.10 Limpieza (Protocolo Obligatorio)

⚠ Limpieza de recursos

Ejecutar al finalizar:

```
deactivate
rm -rf /tmp/codex-lab09

ls /tmp/codex-lab09 2>/dev/null \
  && echo 'ERROR: aún existe' \
  || echo 'OK: limpio'
```

8. Resumen y siguientes pasos

Concepto	Detalle
Pirámide de tests	Unit > Integración > Smoke > Seguridad > E2E. Codex genera en todos los niveles
Prompt para testing	Scope explícito + edge cases + convenciones + ejecutar
Patrón de debugging	Repro steps + expected vs actual + constraints + verification
Test-first debugging	Test que FALLA → fix → test PASA
Auto-fix CI	codex exec en GitHub Action para corregir tests fallidos
SAST + Codex	Complementar scanners con interpretación y patches
Tests de caracterización	Capturar comportamiento actual antes de refactor
AGENTS.md para testing	Reglas de testing y debugging consistentes
Tests placebo	Evitar: test the contract, not the implementation
Flakiness E2E	Waits explícitos, datos deterministas, data-testid

8.2 Preparación para el Módulo 10

En el Módulo 10 abordaremos Automatización y Trabajo en Equipo: codex exec en CI/CD, Codex Cloud, skills personalizadas compartidas y workflows multi-agente.

- Revisar: <https://developers.openai.com/codex/noninteractive/>
- Revisar: <https://developers.openai.com/codex/cloud/>
- Revisar: <https://developers.openai.com/codex/guides/agents-sdk/>

9. Referencias

- Workflows (testing/debugging): <https://developers.openai.com/codex/workflows/>
- Prompting guide: https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide/
- Non-interactive (codex exec): <https://developers.openai.com/codex/noninteractive/>
- Auto-fix CI: <https://developers.openai.com/cookbook/examples/codex/autofix-github-actions/>
- SAST GitLab: https://developers.openai.com/cookbook/examples/codex/secure_quality_gitlab/
- Slash commands (/review): <https://developers.openai.com/codex/cli/slash-commands/>
- GitHub Action: <https://developers.openai.com/codex/github-action/>
- Automations: <https://developers.openai.com/codex/app/automations/>